Índice

1. Validación de Historias de Usuario del Sprint 1

2. Arquitectura del Sistema SIGE-ALM

3. Modelo de Datos del Sistema

1. Validación de Historias de Usuario del Sprint 1

Proyecto SIGE-ALM – Francisco Espada Esparcia

A continuación, se detalla el proceso de validación (testing) realizado para cada Historia de Usuario del Sprint 1, incluyendo las pruebas efectuadas, criterios de aceptación, resultados y evidencias.

HU-01 – Crear artículo

Estado: No implementada en este Sprint.

Validación:
No procede. Se planificará y validará en el Sprint 2 junto con el CRUD completo de artículos.

Se ha creado un articulo de manera manual para poder trabajar y hacer pruebas.

Opciones extra

	id	codigo	nombre	descripcion	stock_actual	stock_minimo	activo	id_categoria	id_proveedor_preferente	created_at	updated_at
Editar Copiar Borrar	1	ART-001	Taladro demo	Taladro percutor	110	10	1	1	1	NULL	2025-12-09 10:15:23

Seleccionar todo *Para los elementos que están marcados:* Editar Copiar Borrar Exportar

HU-02 – Consultar artículos

Estado: No implementada en este Sprint.

Validación:
Pendiente de implementación en el Sprint 2.

HU-10 – Registrar entrada

Estado: COMPLETADA

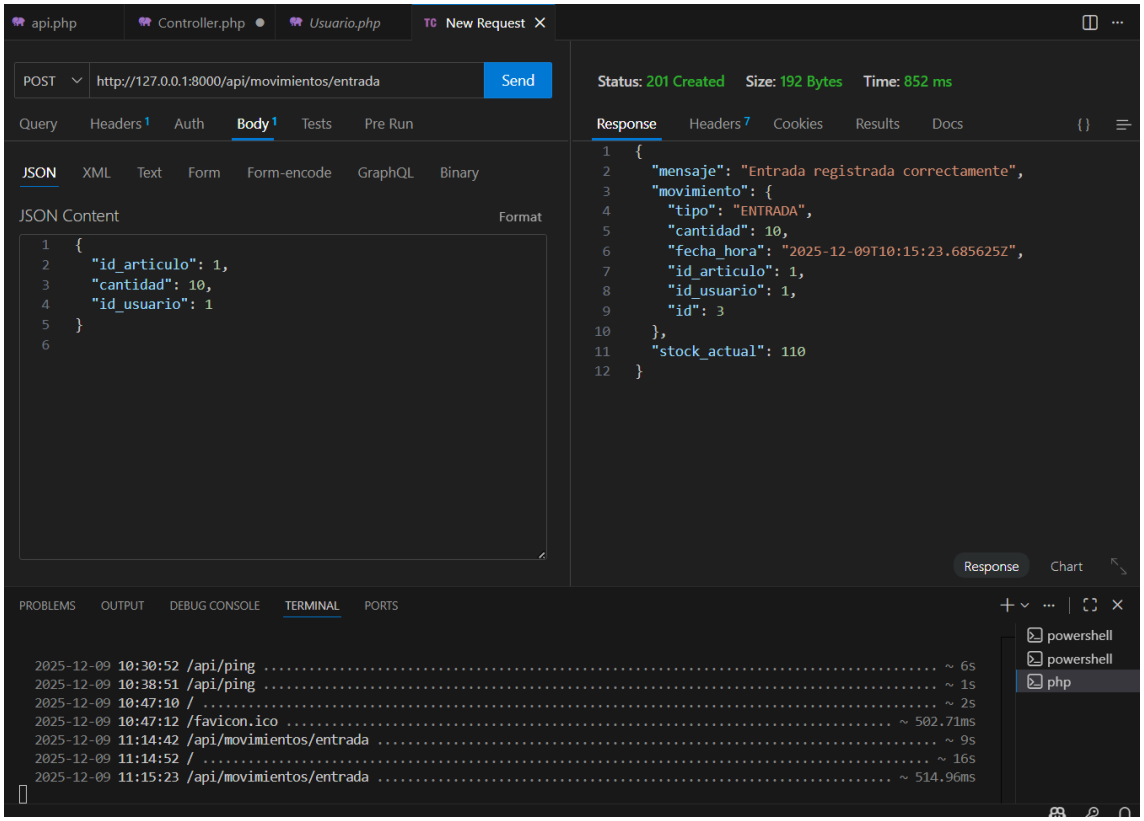
Criterios de aceptación

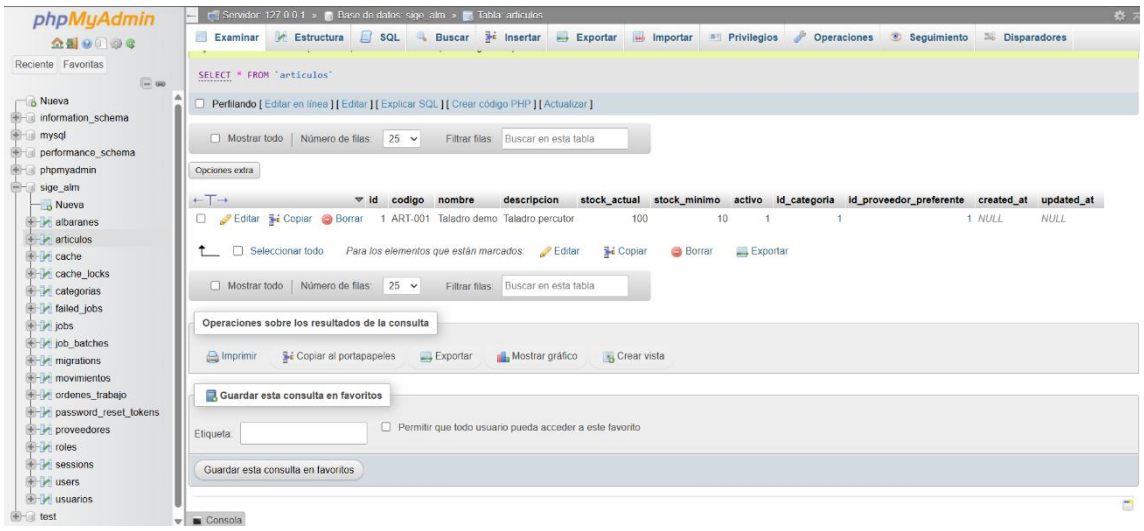
- El sistema debe permitir registrar una entrada de material indicando artículo, cantidad y usuario.
- La cantidad debe sumarse al stock actual del artículo.

- El movimiento debe quedar trazado en la BD.
- La respuesta debe devolverse en formato JSON.

Método de prueba

- Thunder Client (POST /api/movimientos/entrada).
- Verificación del registro en BD MySQL.





HU-11 – Registrar salida por OT

Estado: COMPLETADA

Criterios de aceptación

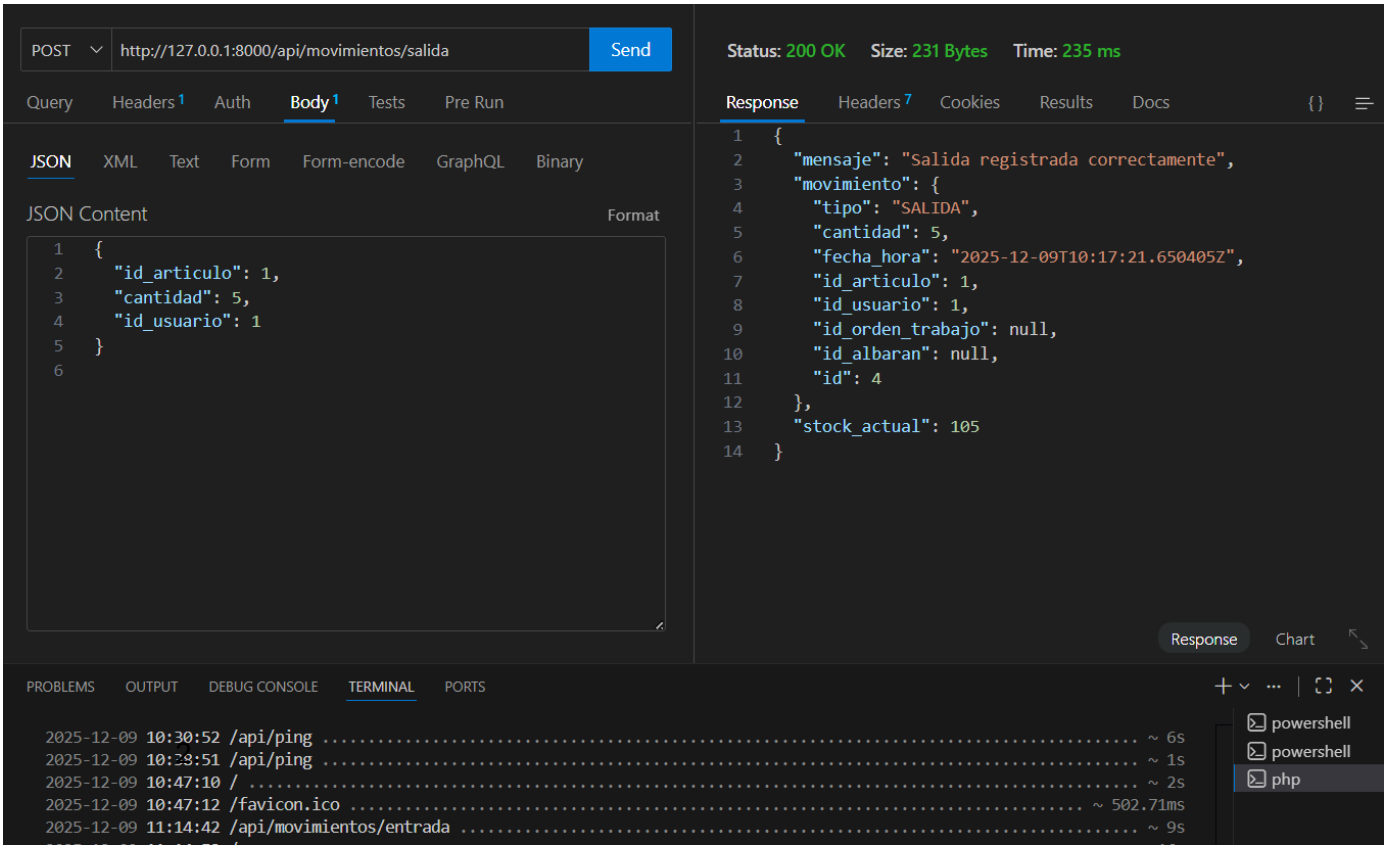
- El sistema debe permitir registrar una salida vinculada a un artículo.
- Debe validar stock suficiente antes de descontar unidades.
- Si no hay stock, debe devolver error sin modificar la BD.
- El movimiento debe quedar registrado como SALIDA.

Método de prueba

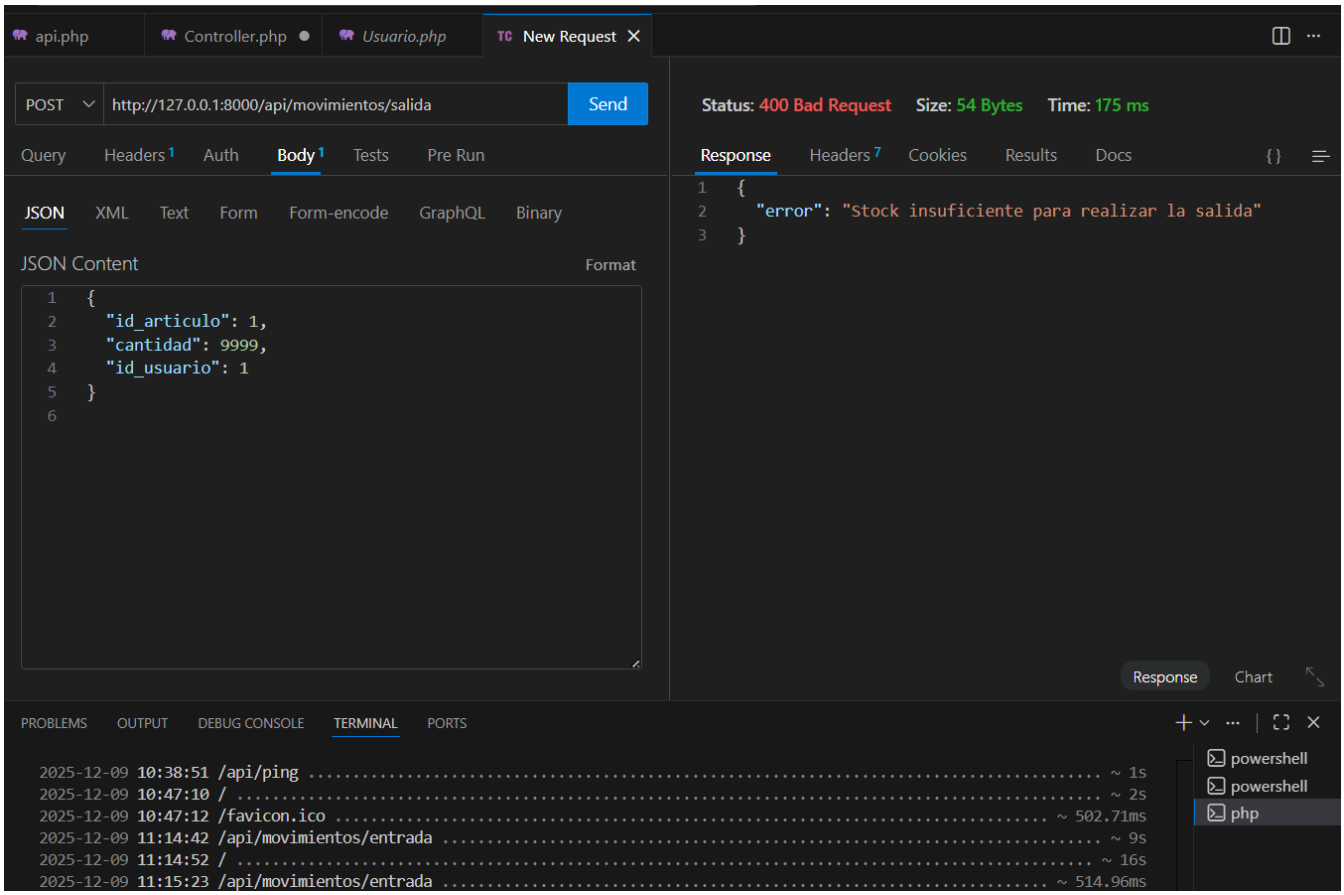
- Thunder Client (POST /api/movimientos/salida).
- Pruebas de casos correctos e incorrectos.
- Auditoría en MySQL para comprobar que el stock no cambia cuando no procede.

Evidencias

1. **Salida correcta**



2. **Salida con error por stock insuficiente**



HU-12 – Registrar devolución

Estado: No implementada

Validación: Pendiente de desarrollo en Sprint 2.

HU-13 – Consultar historial

Estado: PARCIALMENTE completada

Criterios de aceptación

- El sistema debe permitir consultar los movimientos de un artículo u operación.
- Debe mostrar trazabilidad completa.
- Cada operación debe reflejar tipo, fecha, cantidades y usuario.

Método de prueba

Aunque aún no existe endpoint dedicado, se validó la **trazabilidad real**, comprobando:

- Que cada entrada y salida genera un registro en la tabla movimientos.
- Que se almacenan tipo, cantidad, fecha/hora y usuario.
- Que los datos son coherentes con las peticiones realizadas.

Evidencias

Se verificó la trazabilidad validando directamente los registros de la tabla movimientos tras cada operación.

			id	tipo	cantidad	fecha_hora	id_articulo	id_usuario	id_orden_trabajo	id_albaran	created_at	updated_at
Editar	Copiar	Borrar	3	ENTRADA	10	2025-12-09 10:15:23	1	1	NULL	NULL	NULL	NULL
Editar	Copiar	Borrar	4	SALIDA	5	2025-12-09 10:17:21	1	1	NULL	NULL	NULL	NULL

.

HU-14 – Consultar stock

Estado: COMPLETADA

Criterios de aceptación

- El sistema debe devolver el stock actual de un artículo.
- El dato debe actualizarse según las entradas y salidas previas.
- Respuesta en JSON.

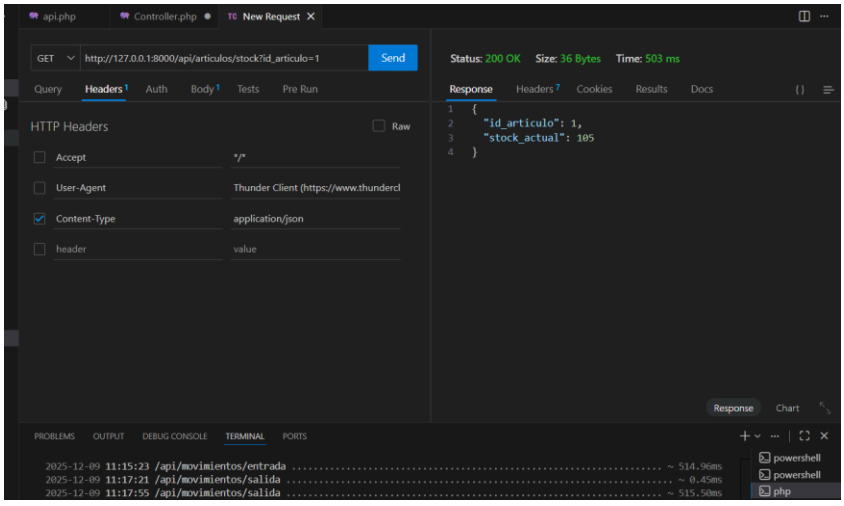
Método de prueba

- Thunder Client (GET /api/articulos/stock?id_articulo=1).
- Comparación del dato recibido con los registros en BD tras movimientos previos.

Evidencias

En las pruebas:

- Se realizaron entradas y salidas.
- La consulta devolvió el stock correcto (105 unidades) tras las operaciones.



HU-20 – Crear orden de trabajo (OT)

Estado: No implementada.

Validación:
Pendiente de desarrollo en Sprint 2.

HU-21 – Consultar OT

Estado: No implementada.

Validación:
Pendiente de desarrollo.

HU-22 – Actualizar estado de la OT

Estado: No implementada.

Validación:
Pendiente de desarrollo en Sprint 2.

HU	**Estado**	**Validación**
HU-10 Registro entrada	COMPLETADA	Validada con pruebas reales
HU-11 Registro salida	COMPLETADA	Validada con pruebas reales
HU-14 Consultar stock	COMPLETADA	Validada con pruebas reales
HU-13 Historial	PARCIAL	Trazabilidad OK, falta endpoint
HU-01 Crear artículo	PENDIENTE	No implementada
HU-02 Consultar artículos	PENDIENTE	No implementada
HU-12 Registrar devolución	PENDIENTE	No implementada
HU-20 Crear OT	PENDIENTE	No implementada
HU-21 Consultar OT	PENDIENTE	No implementada
HU-22 Actualizar estado OT	PENDIENTE	No implementada

2. Arquitectura del sistema

El sistema SIGE-ALM se ha desarrollado bajo una arquitectura en tres capas, separando claramente el acceso a datos, la lógica de negocio y la interacción con el cliente.

Capa cliente

La capa cliente es el punto de interacción con el sistema. En la versión actual se utilizan herramientas como Thunder Client para enviar solicitudes HTTP a los distintos endpoints expuestos por la API. Aunque el sistema aún no cuenta con una interfaz visual definitiva, la capa cliente permite probar la funcionalidad, validar el comportamiento del backend y simular el futuro acceso desde aplicaciones web o móviles.

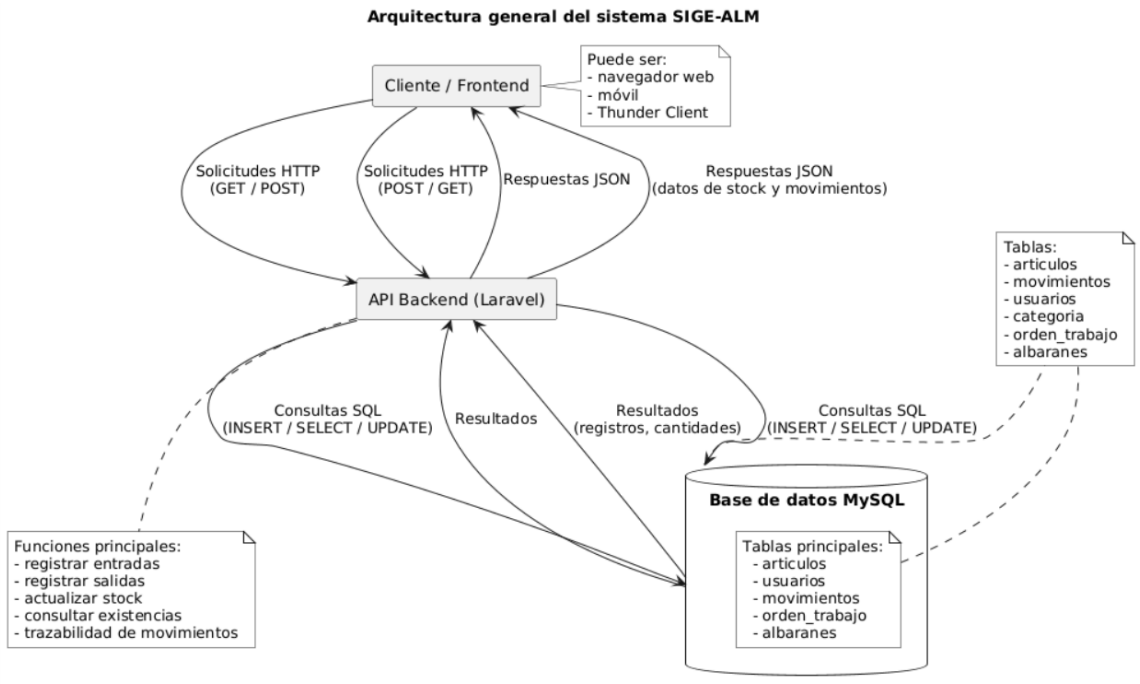
API Backend (Laravel)

La lógica de negocio del sistema se implementa mediante una API REST desarrollada en Laravel. Esta API se encarga de recibir los datos del cliente, validar la información, ejecutar las operaciones correspondientes en la base de datos y devolver respuestas estructuradas. Entre sus responsabilidades principales se encuentran el registro de entradas y salidas, la actualización automática del stock disponible, la persistencia de movimientos y la comprobación de disponibilidad antes de realizar una salida.
La API permite trabajar de forma desacoplada respecto al cliente y facilita la integración con nuevas interfaces o servicios externos sin modificar la estructura interna del sistema.

Base de datos MySQL

La base de datos almacena la información estructurada de artículos, usuarios, movimientos y documentos asociados. La persistencia de datos está completamente gestionada mediante operaciones SQL realizadas desde la API, garantizando integridad, trazabilidad y consistencia del inventario.

La actualización del stock se realiza automáticamente como resultado de cada operación registrada, evitando manipulaciones manuales y minimizando errores. La arquitectura facilita que cualquier consulta sobre el inventario, el histórico o la actividad almacenada pueda resolverse directamente desde la API.



3. Modelo de datos (base de datos)

Tablas principales

La base de datos del sistema SIGE-ALM se ha diseñado a partir del diagrama de clases definido en la Actividad 1.

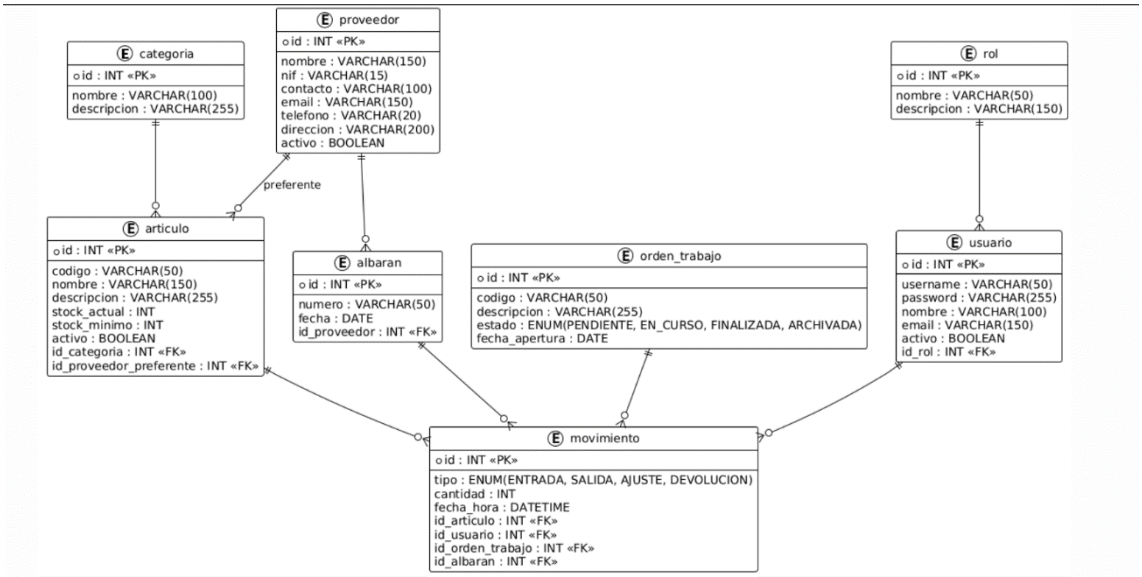
El modelo relacional resultante incluye las tablas categoría, proveedor, rol, usuario, articulo, orden_trabajo, albarán y movimiento, que permiten cubrir los casos de uso seleccionados: registro de entradas por albarán, registro de salidas por OT y consulta de stock y trazabilidad.

La tabla movimiento centraliza la lógica de actualización de inventario mediante el campo tipo (ENTRADA/SALIDA/AJUSTE/DEVOLUCION) y sus claves foráneas hacia artículo, usuario, orden_trabajo y albarán, lo que garantiza una trazabilidad completa de cada operación.

Relaciones entre entidades

Las entidades del modelo están conectadas a través de claves externas que vinculan cada movimiento con el artículo afectado y con el usuario que lo ejecuta. El artículo puede estar relacionado con una categoría y un proveedor, y los movimientos pueden asociarse opcionalmente a documentos operativos como órdenes de trabajo o albaranes.

Estas relaciones permiten que todas las operaciones queden registradas dentro de un contexto documental claro, favoreciendo auditoría del almacén y evitando duplicidad de información.



Restricciones (claves, integridad referencial)

El modelo implementa claves primarias en cada entidad y claves externas entre movimientos, artículos, usuarios y documentos relacionados. Gracias a la integridad referencial, el sistema evita registrar movimientos sobre artículos inexistentes o usuarios no válidos, y asegura que cada operación tenga un origen consistente.

Además, las restricciones sobre cantidades impiden valores negativos y garantizan que el stock refleje la suma real de todas las entradas y salidas registradas. Con esto, la base de datos actúa como un punto único y fiable para toda la lógica del almacén.